Mobile Robot Navigation in Structured Environments with Visual Target Tracking

Jan Čejka[1], William Balucha[2], Filip Hladký[3]

***Abstract*—This paper presents a fully integrated solution for autonomous mobile robot navigation in a structured, semi-known indoor environment using visual target tracking. The task requires a TurtleBot platform to exit a garage, detect and circumnavigate a visually defined target, and subsequently return to the garage. The proposed approach combines lightweight color-based image segmentation with depth sensing and closed-loop odometry control within a finite state machine framework. Experimental evaluation demonstrates reliable task execution within the specified time constraints, with robust perception and sufficient odometric accuracy for local navigation. The results show that the proposed system achieves high reliability while maintaining low computational complexity, making it a practical alternative to more resource-intensive navigation methods in semi-known environments.**

***Index Terms*—mobile robot navigation, visual target tracking, finite-state machine, autonomous systems, path planning, odometry evaluation**

I. Introduction

AUTONOMOUS navigation in structured indoor environments is a fundamental problem in mobile robotics because it combines perception, localization, planning, and control. This work presents an onboard solution for a TurtleBot platform tasked with leaving a garage, detecting a green pylon, circumnavigating it without contact, and returning to the garage.

The task is defined in a semi-known environment containing one garage and one pylon, while the robot's initial orientation and the relative placement of the pylon may vary between runs. The robot uses RGB images, depth measurements, point-cloud data, and odometry to estimate the pylon position, control motion, and complete the full sequence autonomously.

A. Problem Formulation

This paper focuses on Task 3: The robot encircles the pylon and parks. Here, the robot needs to find the exit of the garage and drive out of it. The pylon is placed in a random position in the environment. The robot needs to locate the pylon, drive up to it on its own and encircle it. After this manoeuvre it needs to drive back to the garage and park without coming into contact with the surrounding objects all throughout its movement. The robot must complete the task autonomously in the time limit of 8 minutes.

Authors are with the Faculty of Electrical Engineering, Czech Technical University in Prague, Prague, Czech Republic.

[1] e-mail: cejkaja5@cvut.cz

[2] e-mail: balucwil@cvut.cz

[3] e-mail: hladkfil@cvut.cz

Manuscript received April 13, 2026.

Assumptions - Pylon is a green ball with a fixed diameter of $\approx 90\ \mathrm{mm}$ and is placed between $0.8\ \mathrm{m}$ from the closest garage wall and $5\ \mathrm{m}$ from the centre point of the garage. It is assumed, that there is enough room to drive around the pylon and around the garage. The garage is a three sided box, with an opening wide $\approx (485 \pm 20)\ \mathrm{mm}$, with yellow coloured walls and two purple strips on each side of the opening.

B. System and Environment Description

The solution is implemented on the TurtleBot platform equipped with Intel RealSense D435/D435i sensor, which provides synchronized RGB images and 3D depth data at a resolution of 640 $\times$ 480 pixels. The robot is also equipped with odometry, a front bumper, and programmable buttons. On board of TurtleBot is Robot Operating System and Singularity virtualization toolbox. Two free-standing objects in this environment are a yellow garage with purple front entrance pillars and a green ball on a plastic stand serving as a model of a pylon. The guardrail enclosing the available space is made up of brown OSB approximately 30 cm high.

C. Contribution

While many modern autonomous navigation systems rely on computation-heavy algorithms like SLAM or big CNNs [1], [2], this paper presents a lightweight, robust alternative tailored for semi-known environments. Our primary contribution is the development of a fully integrated, state-machine-pipeline that successfully combines computer vision with closed-loop odometry control. By utilizing local depth heuristic and dynamic path planning rather than global (layered) costmap [3], the proposed algorithm achieves high reliability and low latency in target acquisition, 2D navigation and collision avoidance.

II. Our Approach

A. System Overview

The proposed solution is comprised of sequential pipeline consisting of perception, localization and motion-control units. See Fig. 1 with the high-level structure of the implemented algorithm. The architecture combines reactive perception for target acquisition with closed-loop odometry control for local motion execution. Each stage is comprised of its own control routine, while low-level motion control is performed by odometry-based helpers for rotation and point to point driving. The robot utilizes RGB images for pylon and garage detection, point-cloud data for object distance estimation.

Angular camera offset had to be considered to compensate for the fact that the camera did not point straight ahead.

```
def run(self) -> None:
    self.exit_garage()
    self.approach_pylon()
    self.drive_around_pylon()
    self.return_to_garage()
```

Fig. 1. Code snippet describing discussed algorithm.

B. State System

The behaviour of the robot is modelled as finite-state machine, with 5 major states as shown in figure 2.

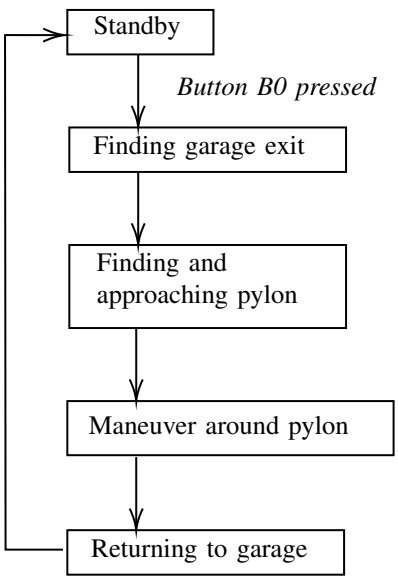

Fig. 2. State space diagram.

III. Visual Target Detection

For object detection, RGB and HSV formats are used. To identify targets in the environment, the image acquired from the robot's onboard RGB camera is converted to HSV colour space to negate the effects of surrounding lighting on the environment. A colour mask is applied on the HSV image to isolate spots within a specific range of colour values. The resulting binary mask is refined using morphological opening and closing operations with a 5×5 kernel to remove noise and fill small gaps. Contours are then extracted from the processed mask. To eliminate false detections and reject background artefacts, several constraints are applied depending on the specific target object.

A. Pylon Segmentation

To identify the Pylon, the robot must find a green ball. The applied colour mask is set to isolate values for the green colour, acquired by testing on sample data.

Each contour must have an area of at least 100 pixels. Furthermore, the circularity of each contour is evaluated to ensure the detected object approximates a circular shape. Circularity is defined as:

$$C = \frac{4\pi A}{P^2} \tag{1}$$

where A is the contour area and P is its perimeter. Only contours with circularity greater than 0.45 are considered valid candidates.

For each valid contour, the minimum enclosing circle is computed and its centre is used as the representative position of the detected object. The contours are processed in descending order of area, ensuring the largest contour is selected as the primary target, as it is assumed that it is the closest pylon to the robot.

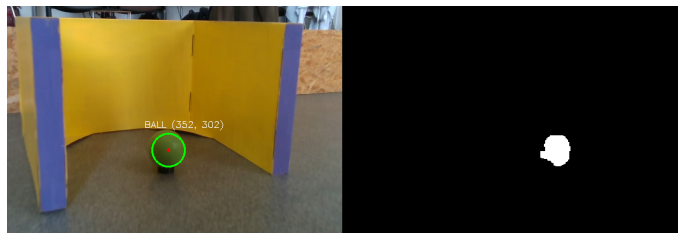

Fig. 3. Detected pylon (left) and corresponding binary mask after HSV thresholding (right).

B. Purple Pillar Detection

To identify the garage entrance, the robot must find two purple pillars. Here, the colour mask is applied to isolate the purple hue.

Each contour has to have an area of at least 600 pixels and exhibit a contour solidity of at least 0.8. Contour solidity is a measure of hollowness of the rectangle, described with the following equation:

$$\frac{A}{\mathcal{A}} \geq 0.8, \tag{2}$$

where A is the area of the contour and $\mathcal{A}$ is the area of the convex hull, the smallest convex set containing the original contour. Additionally, due to the physical structure of the pillars, the found contour has to fulfil an aspect ratio check, where its ratio of width to height has to be less than 0.5.

For greater robustness, the centre of every contour is computed as the image moment of the original contour, enabling the segmentation not to be affected by sudden concealment of the pillar. Valid centres of the contours are sorted by their distance to the camera centre to focus only at one pillar at a time.

Fig. 4. Detected purple pillars (left) and corresponding binary mask after HSV thresholding (right)

IV. Exit Garage and Drive Around Pylon

As specified in the task description, the robot's initial position and orientation within the garage are random. Robot's home point is defined as a point in front of the garage, 0.7 m away from garage centre (see Fig. 6). The robot first identifies the exit and subsequently drives to the home point to leave the garage. This distance was empirically determined to place the robot in an optimal position for reliable pylon detection.

A. Exit Garage

To estimate the wall positions, point-cloud data are utilised. The processing pipeline consists of the following steps. First, the point cloud is sliced to extract the y and z axes, where y denotes the lateral position and z represents depth, invalid values like NaN or infinity are removed. The 10th percentile of the depth distribution was used to suppress noise and hallucinated measurements.

Odometry was used for rotation and yaw recording. As described in Fig. 5 heuristic scanning for walls is implemented. The robot starts rotating counter-clockwise. When free space is detected—i.e., when the point-cloud distance exceeds the threshold of $0.5\,\mathrm{m}$—the detection of the first wall is initiated. When the measured distance reaches value smaller than the free space threshold robot immediately stops and records yaw ψ_1 of the first wall. Thereafter robot starts rotating clockwise until it detects second wall whose yaw ψ_2 it also stores.

The fail-safe condition

$$\psi_1' - \psi_2' > 0.75\,\mathrm{rad}, \tag{3}$$

where $\psi_{1,2}'$ represent normalised angles as described in Section VI-D, is used as a check that the robot does not detect the same wall twice due to sensor noise. Bisecting angle is taken as a normalised value of ψ_{mid}, where

$$\psi_{\mathrm{mid}} = \begin{cases} \frac{\psi_1+\psi_2}{2}, & \text{if } \psi_1 \geq \psi_2, \\ \frac{\psi_1+\psi_2}{2} + \pi, & \text{if } \psi_1 < \psi_2. \end{cases} \tag{4}$$

The robot rotates towards ψ_{mid} and sets this position as the origin of the global coordinate frame.

B. Pylon Detection

Since the pylon can be positioned anywhere within the enclosed environment, a systematic search strategy is required.

1) Pylon Localization Manoeuvre: The robot's search trajectory originates at the home point and proceeds through four designated points near the garage, coupled with five conditional waypoints. These conditional points are only visited if the robot confirms a clear path (e.g., no obstructing walls); otherwise, they are bypassed to ensure efficiency. This extended path allows the robot to locate the pylon even when positioned at the perimeter of the environment. Based on the assignment specifications, the area immediately surrounding the garage is assumed to be an obstacle-free "safe zone". The complete trajectory is illustrated in Fig. 6. At each waypoint, the robot performs a full 360° rotation to scan for the pylon using RGB image data. If the pylon is detected, the robot immediately terminates the search pattern and manoeuvres toward the target; otherwise, it proceeds to the next point in the sequence to continue the search.

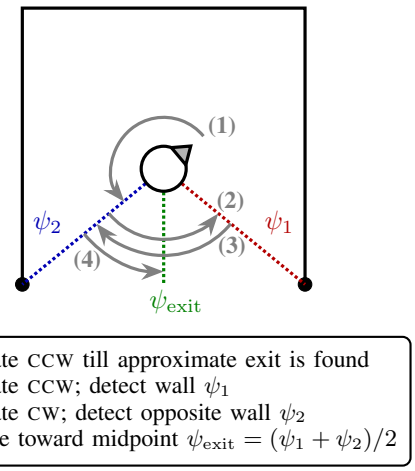

Fig. 5. Schematic of `find_exit` logic. Circle represents the robot with heading given by its gray beak.

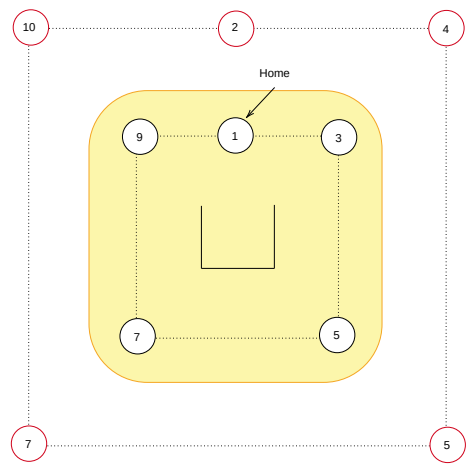

Fig. 6. The robot's trajectory points during the pylon search. Red circles indicate conditional waypoints, while the orange zone represents the safe area around the garage. Not to scale.

C. Driving Towards Pylon

When the pylon is detected using RGB data, the robot begins to approach it at a low speed. To follow the pylon smoothly, a P-regulator is utilized to control the robot's angular velocity, which is proportional to the pylon's horizontal offset from the center of the camera frame. If the robot loses sight of the pylon at any point, it reverts to the localization maneuver.

To prevent collisions, the robot monitors whether the garage is dangerously close to its path toward the pylon. If an imminent collision is detected, this is accounted for when computing the angular velocity, directing the robot away from the garage. To determine if the garage obstructs the robot's path, the garage's position (the origin of the global coordinate system) is transformed into the robot's local coordinate frame.

The resulting angular velocity is computed as

$$\omega = -0.001 \cdot e_{\mathrm{pixel}} + \mathrm{sgn}(x_{\mathrm{origin}}) \cdot \left(\frac{0.8 - |x_{\mathrm{origin}}|}{0.8} \right) \tag{5}$$

where e_{pixel} be the error in the image plane, defined as $(\text{column} - 320)$ and x_{origin} be the local x-coordinate of the garage. Second product in this equation is applied only if robot is in so called "danger zone", illustrated by blue area in Fig. 7.

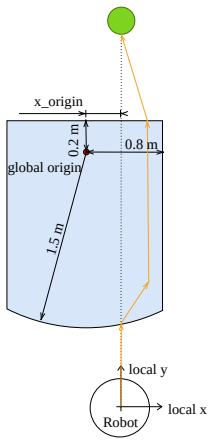

Fig. 7. "Danger zone", where the robot is being directed away from the garage, and its effect on the robot's path to the pylon.

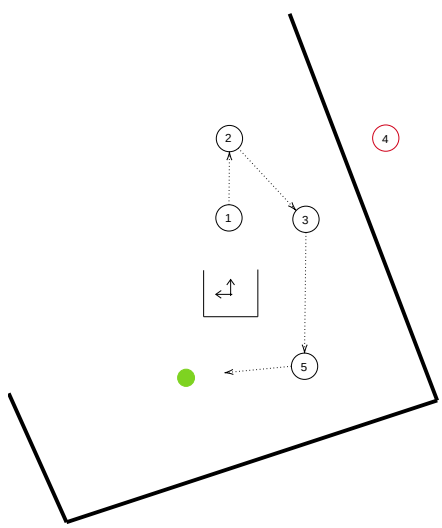

Fig. 8. An example run of the search algorithm. Note that the robot bypassed conditional waypoint 4 because it is blocked by the environment's boundary wall.

D. Pylon Circumnavigation

Once the robot detects that the pylon is $80\,\text{cm}$ away, it continues the approach using odometry, as the pylon is no longer visible in the camera frame, until reaching the target distance of $35\,\text{cm}$. After that, it executes hard-coded maneuver to drive around pylon. Defined trajectory is illustrated in Fig. 9. Established points are converted to global coordinates using affine transformation described in VI-B.

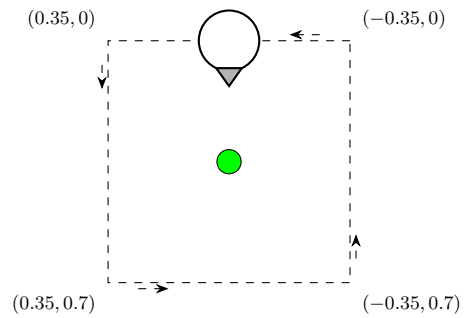

Fig. 9. Diagram illustrating the trajectory robot takes when encircling the pylon, drawn green. Marked coordinates are in the local frame $\mathcal{F}_r$ while their respective values in $\mathcal{F}_g$ are sequentially commanded to the robot.

V. Return to the Garage and Parking

This part is comprised of three main sequential methods. The main overview can be found in Fig. 10. The starting assumption is that the robot has already successfully encircled pylon and has recorded its trajectory up to this point.

```
def return_to_garage(self) -> None:
    self.approach_garage():
    self.find_garage_entrance():
    self.drive_into_garage():
```

Fig. 10. Return to garage overview.

In the following sections we will describe return to garage pipeline.

A. Approaching Garage

The robot's navigational graph is constructed using known safe coordinates. Based on the specified 0.8 m obstacle-free zone around the garage, points 1, 3, 5, 7 and 9 from Fig. 6 are initially designated as safe. Additionally, any conditional way-points visited by the robot during the pylon search and points visited during pylon circumnavigation are added to this safe set. Using these points as vertices, a graph is generated where edge weights represent the Euclidean distance between them. To prevent collisions, an edge-pruning mechanism is applied: any edge intersecting the obstacle boundaries—defined as a circle with $R = 0.69$ m for the garage and $R = 0.3$ m for the pylon—is completely removed from the graph. Because safe points form a continuous boundary around both the garage and the pylon, a valid, collision-free route is guaranteed. Finally, Dijkstra's algorithm [4] is applied to the graph to determine the shortest possible path back to the home point.The Dijkstra's algorithm was implemented using the NetworkX [6] library. Figure 11 shows a visualization of the travelled route (red arrows), safe points (black circles), exclusion zones (orange circles), and the calculated return path (blue arrows).

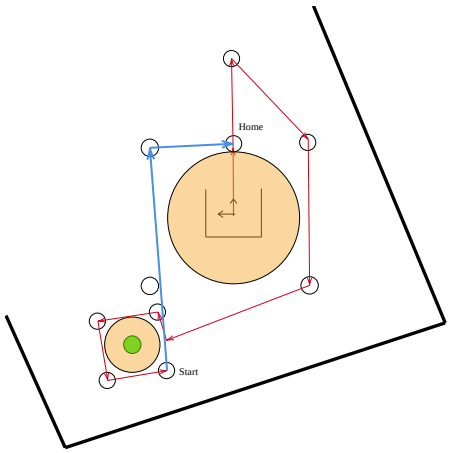

Fig. 11. An illustration of a robot's route back to its home position.

To carry out this task odometry navigation function described in Section VI-A is utilised, placing the robot approximately $\pm 10\,\mathrm{cm}$ in either direction from home point before the garage entrance. The discrepancy in final robot position is negligible, because it serves just as an initial attempt at locating the garage and further logic is handled by a more sophisticated method. The entire approach garage method is showcased in Fig. 12.

```
def approach_garage(self) -> bool:
    for point in self._get_path_to_garage():
        if not self._go_to_point_using_odometry
        ↪  (*point):
            return False
    return self._rotate_to_angle(math.pi)
```

Fig. 12. Code snippet describing garage approaching logic.

B. Finding Garage Entrance

Once the robot has returned back to the approximate vicinity of the garage using odometry, it starts a $(2\pi + 0.2)$ rad rotational sweep to find the global position of the pillars. Image segmentation of garage pillars is described in Section III-B.

Continuous motion of the camera often degrades the depth estimation due to sensor noise. To mitigate this issue, robot continuously tracks the segmented pillars in the RGB frame. When, during rotation, pillar centres approach optical centre of the camera to the distance of 100 pixels, robot halts its angular velocity and wait for a fresh set of RGB, depth and odometry data. This step is crucial, because otherwise the RGB frame received at a different time than the point cloud frame would not align correctly. The depth of the the pillar, i.e. its distance from robot, is calculated as the average 3D point in a window around pillar centre, acquired by a method described in Section VI-E.

Once stationary, the depth is estimated by averaging valid points in the neighbourhood of the detected pixel centre (u, v) to filter out depth-sensor noise. The resulting local vector $\vec{p}_{\text{local}} = [x, z]^T$ is adjusted for a known camera angular offset of $0.1\,\mathrm{rad}$ to compensate for mounting misalignment. Finally, the local coordinates are transformed into the global odometry frame $\mathcal{F}_g$ using the affine transformation described in equation number 9. The robot then resumes its rotation to locate the second pillar, ensuring that duplicate detections of the same pillar are filtered by a threshold-based yaw check of 0.4 rad.

C. Gate Alignment

Let the global coordinates of the pillars be $P_1 = (x_1, y_1)^T$ and $P_2 = (x_2, y_2)^T$. The garage entrance is assumed to be the geometric mean of the pillar centres, i.e.

$$G = \left(\frac{x_1 + x_2}{2}, \frac{y_1 + y_2}{2} \right)^T. \tag{6}$$

To park successfully robot must not only find the garage entrance G, but do so facing the correct direction. The angle ϱ between system basis and directional vector marked with green in Fig. 13, is calculated as follows

$$\varrho = \operatorname{atan2}(dx, -dy), \tag{7}$$

where dx, dy are vector components of $\vec{p}_{12}$ in the system $\mathcal{F}_g$, as portrayed in the Fig. 13. Function $\operatorname{atan2}$ is defined as in [5]. To ensure that the robot is facing inward rather than outward, additional check against current robot's heading direction ϱ_{curr} is performed. If their difference is greater than $\frac{\pi}{2}$, then angle of $\pi\,\mathrm{rad}$ is added to ϱ. Finally the robot employs odometry-based controller described in Section VI-A to successfully stage itself for a final parking manoeuvre.

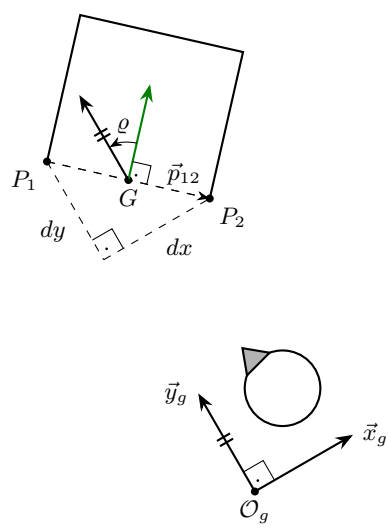

Fig. 13. Figure depicting geometric relationships established during the `find_garage_entrance` routine. The spatial discrepancy $\vec{p}_{12}$ between pillars P_1 and P_2 is decomposed into components dx and dy relative to the global odometry frame $\mathcal{F}_g = (\mathcal{O}_g, \langle \vec{x}_g, \vec{y}_g \rangle)$. The entrance normal, marked green, is computed orthogonally to $\vec{p}_{12}$, determining the correct heading target for the robot.

D. Driving into the Garage

Once the robot is positioned at point G and aligned with the target heading, the final parking sequence initiates. Similar to the `find_exit` method described in Section IV-A, the robot utilizes point-cloud data for depth estimation.

To focus exclusively on the garage's back wall and ignore peripheral obstacles or background noise, a central Region of Interest spanning 80×80 spatial points is extracted from the

depth matrix. The point cloud is further masked to filter out the floor plane and distant outliers. As in the exit sequence, the 10th percentile of the filtered depth array is used to compute a robust, noise-resistant distance to the wall.

During the approach, the robot's forward translation is continuously adjusted in real time. A proportional controller minimizes the angular heading error to compensate for odometry drift, ensuring the robot drives straight into the enclosure. The robot advances until the depth array registers a proximity of $34\,\mathrm{cm}$ from the back wall. Upon reaching this threshold, the robot halts its movement, successfully concluding the parking manoeuvre.

VI. Foundational Control and Algorithmic Framework

The execution of the high-level navigation logic relies upon a set of core functional building blocks described herein. This framework includes low-level proportional controllers for precise movement and mathematical utilities for frame-to-frame transformations, ensuring the stability and accuracy of the robot's spatial reasoning.

A. 2D Navigation Utilizing Odometry

The goal of this section, implemented in `_go_to_point_using_odometry`, is to navigate the robot to a target 2D point using a two-phase odometry-based strategy. The navigation consists of

1) rotating the robot to face the target point,
2) driving towards the target while maintaining alignment.

This process relies entirely on odometry feedback and employs helper methods for rotation and translation. The function exits early if any step fails or if execution is interrupted.

1) Target Bearing Sequence: The heading yaw to the target point $\vec{p}_{\mathrm{dest}}$ from the current position $\vec{p}_{\mathrm{curr}}$ in the global frame $\mathcal{F}_g$ is computed from their coordinate difference. Let

$$\vec{p}_{\mathrm{curr}} = \begin{pmatrix} x_{\mathrm{curr}} \\ y_{\mathrm{curr}} \end{pmatrix}, \quad \vec{p}_{\mathrm{dest}} = \begin{pmatrix} x_{\mathrm{dest}} \\ y_{\mathrm{dest}} \end{pmatrix}.$$

Then, the heading yaw ϱ is computed as

$$\varrho = \mathrm{atan2}\big(y_{\mathrm{dest}} - y_{\mathrm{curr}},\, x_{\mathrm{dest}} - x_{\mathrm{curr}}\big). \tag{8}$$

The resulting angle ϱ is subsequently normalized as discussed in Section VI-D.

After the heading yaw ϱ is acquired, closed-loop proportional controller is used to smoothly approach the target angle while reducing speed near the goal. Implemented P-controller is illustrated in Fig. 14. Minimal angular velocity was clamped at $\pm 0.2\,\mathrm{rad}\cdot\mathrm{s}^{-1}$ to compensate for motor deadband caused by static friction. The physical system acts as a integrator converting the commanded angular velocity to the robot's yaw. If the current yaw with the desired yaw differ by the angle smaller than $0.05\,\mathrm{rad}$, robot halts its movement and proceeds to the second part of the sequence.

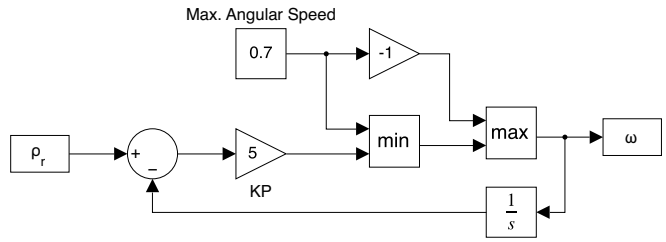

Fig. 14. Simulink diagram illustrating closed-loop proportional control architecture with speed clamp of $[-0.7, 0.7]\,\mathrm{rad}\cdot\mathrm{s}^{-1}$ and proportional gain $K_P = 5\,\mathrm{s}^{-1}$. Input signal is reference yaw ϱ_r and output signal represents angular speed ω. Block $\frac{1}{s}$ represents Laplace transform for integration, not to be confused with seconds.

2) Translation sequence: Once the robot has reached the correct yaw, it will start approaching the point $\vec{p}_{\mathrm{dest}}$. In the method `_drive_to_point` robot continuously adjusts its heading using a proportional angular controller to stay aligned with the target point while moving forward.

The baseline forward velocity was chosen to be $0.2\,\mathrm{m}\cdot\mathrm{s}^{-1}$ to mitigate odometry drift caused by wheel slippage and other factors. If the heading error becomes too large ($> 0.5\,\mathrm{rad}$), the robot temporarily stops forward motion and rotates in place to correct its orientation. P-controller responsible for real-time heading adjustment is the same one as the one illustrated in Fig. 14.

When the robot approaches the point $\vec{p}_{\mathrm{dest}}$ within a Euclidean distance of $8.5\,\mathrm{cm}$, all motion is stopped, concluding the translation to the desired point. This corresponds to a position error of up to $8.5\,\mathrm{cm}$ in either direction. The tolerance was empirically chosen to balance positional precision and algorithmic robustness. A stricter tolerance could cause the robot to miss the target and overshoot, whereas a larger one would compromise spatial accuracy.

B. Coordinate Transformations

Conversions between global coordinates $\mathcal{F}_g$ comprised of origin $\mathcal{O}_g$ with basis spanned by vectors $\vec{x}_g, \vec{y}_g$ and local robot coordinates $\mathcal{F}_r = (\mathcal{O}_r, \langle \vec{x}_r, \vec{y}_r \rangle)$ are often necessary to map obstacles and targets detected by the onboard depth camera into the global odometry space for spatial reasoning. Discussed coordinate frames are illustrated in Fig. 15. The following logic is implemented in `local_coords_to_global_coords`, or its inverse method.

Global coordinate frame $\mathcal{F}_g$ follows standard ROS odometry convention defined in REP 103 [7], which specifies a right-handed coordinate system with $\vec{x}_g$ axis being positive forward and $\vec{y}_g$ to the left. Robot's position in this frame is defined by a translation vector $\vec{t} = (\vec{x}_{\mathrm{rob}}, \vec{y}_{\mathrm{rob}})^T$ and its heading yaw θ.

Conversely, the local frame $\mathcal{F}_r$ utilizes computer vision convention [8], where $\vec{x}_r$ axis is positive to the right and $\vec{y}_r$ axis is positive forward. For an accurate transformation of coordinates of point $\vec{p}_r = (\vec{x}_{P,r}, \vec{y}_{P,r})^T$ from local coordinates to the global point $\vec{p}_g = (\vec{x}_{P,g}, \vec{y}_{P,g})^T$, an intermediate permutation of bases is needed before applying the standard affine transformation.

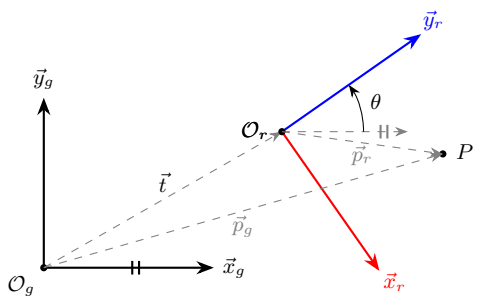

Fig. 15. Two dimensional geometric relationship between the global odometry frame $\mathcal{F}_g = (\mathcal{O}_g, \langle \vec{x}_g, \vec{y}_g \rangle)$ and the robot's local body frame $\mathcal{F}_r = (\mathcal{O}_r, \langle \vec{x}_r, \vec{y}_r \rangle)$. A local point observation $\vec{p}_r$ is mapped to the global point $\vec{p}_g$ via the robot translation vector $\vec{t}$ and rotation by heading θ.

The complete affine transformation is defined by 2D rotation matrix $\mathbf{R}(\theta)$ and permutation matrix $\mathbf{P}$, together producing transformation matrix $\mathbf{T}_{\mathcal{F}_g \to \mathcal{F}_r} = \mathbf{R}(\theta)\mathbf{P}$ from basis $\mathcal{F}_g$ to $\mathcal{F}_r$, written in a compact form

$$\vec{p}_g = \vec{t} + \mathbf{T}_{\mathcal{F}_g \to \mathcal{F}_r} \vec{p}_r = \vec{t} + \mathbf{R}(\theta)\mathbf{P}\vec{p}_r \tag{9}$$

or in expanded form

$$\begin{pmatrix} x_{P,g} \\ y_{P,g} \end{pmatrix} = \begin{pmatrix} x_{\text{rob}} \\ y_{\text{rob}} \end{pmatrix} + \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix} \begin{pmatrix} x_{P,r} \\ y_{P,r} \end{pmatrix}. \tag{10}$$

Equation number 10 was implemented in the method as an algebraic operation to ease computation. Furthermore, it was decomposed into two methods for a better readability.

The inverse operation, transformation of global to local coordinates, is achieved with a simple modification of equation number 10 to the following form

$$\vec{p}_r = \mathbf{T}^{-1}_{\mathcal{F}_g \to \mathcal{F}_r} \left(\vec{p}_g - \vec{t}\right) = \mathbf{P}^{-1}\mathbf{R}(-\theta)\left(\vec{p}_g - \vec{t}\right). \tag{11}$$

C. Safety Handling

Safe operation is achieved by reactive, low-latency interrupt mechanism based on bumper sensor feedback. As per task requirement, any physical contact detected by front bumper must result in an immediate cessation of motion.

This behaviour is achieved by global boolean flag `self.stop`, which acts as system-wide bumper press indicator. The flag is set asynchronously upon bumper callback, and is subsequently checked at the beginning of every control routine and inside of every control loop. This design ensures that safety handling is independent of running method and provides uniform interruption of motion across whole architecture.

Upon activation of `self.stop` flag, robot issues zero-velocity command. The corresponding logic is illustrated in code snippet in Fig. 16. Every method is terminated upon bumper contact, halting any robot movement. The robot rapidly cycles through all remaining states, immediately exiting each one, until it reaches the standby state.

```
if self.stop:
    self.robot.cmd_velocity(0, 0)
    return False
```

Fig. 16. Emergency stop condition triggered by bumper activation.

D. Angle Normalization

Often, subtracting two angles yields a result outside the interval $[-\pi, \pi)$. To address this, a `normalize_angle` function is employed, ensuring robot always takes the shortest path during rotation. A given angle θ is normalized to $\hat{\theta}$ as follows:

$$\hat{\theta} = (\theta + \pi) \bmod (2\pi) - \pi \tag{12}$$

E. Average of Nearby Pixels

Raw 3D point clouds generated by depth sensors are often noisy and do not correctly represent the real measured distance. Thus taking a single pixel for navigation is unreliable, instead, a local neighbourhood of pixels must be analysed for an actual 3D distance.

A sliding window of size 5×5, centred at a target pixel (y, x) where x and y represent pixel column and row coordinates, is introduced to define a local Region of Interest. In first pass candidates for valid points $\mathcal{P}_{\text{valid}}$ are extracted. Each point $\vec{p}_i = (x, y, z)^T \in \mathcal{P}_{\text{valid}}$ if and only if $\vec{p}_i \in \mathbb{R}^3 \setminus \{0, \pm\infty\}$. This step effectively prunes sensor dropouts and leaves clean a dataset.

To prevent "bleeding" of coordinates across different depths (e.g., when window spans both a foreground and background structures), a distance-based refinement is implemented. Ideally using a centre pixel of the window as a reference $\vec{p}_{\text{ref}}$ or a mean value of central column, whenever central pixel is invalid. Set of valid points $\mathcal{P}_{\text{valid}}$ is once again pruned and only those points which satisfy Euclidean distance condition in equation 13 are retained.

$$\|\vec{p}_i - \vec{p}_{ref}\| \leq \delta, \tag{13}$$

where $\delta = 0.1\,\text{m}$.

The final estimated 3D distance $\vec{p}_{\text{avg}}$ is obtained as a mean value from the remaining N filtered points in $\mathcal{P}_{\text{valid}}$:

$$\vec{p}_{\text{avg}} = \frac{1}{N} \sum_{i=1}^{N} \vec{p}_i. \tag{14}$$

If no valid points are obtained, method returns a null result, signalling higher-level logic that the depth data at the current position is unreliable.

VII. Engineering Choices

In order to determine the specific constants used in the segmentation methods, the 10th percentile threshold for point-cloud data, and the odometry tolerance values, we carried out a series of empirical experiments on the target platform and dataset. We iterated through a range of candidate values, evaluated their effect on both stability and responsiveness, and selected the configuration that produced the most reliable behavior.

For the segmentation methods, the parameters were adjusted to consistently detect pylons and garage pillars while avoiding false positives from the surrounding environment. For this tuning process, sample data consisting of hundreds of images captured under various lighting conditions was recorded.

It is worth noting that the robot often appears to drive quite slowly. This is primarily due to the effective update rate of the perception stack: new sensor data are received at a low frequency.

VIII. Controlling the robot

After executing `python3 main.py` within the Singularity container environment, the robot enters a standby mode. To initiate the algorithm, press the B0 button. The robot will return to standby mode when any of the following conditions is satisfied:

1) the robot has successfully parked in the garage,
2) robot didn't find the pylon after visiting all waypoints,
3) the B1 button is pressed,
4) any bumper is pressed.

A short sound is played immediately before the robot switches back to standby mode to indicate whether the execution ended in success or failure.

IX. Experiments

A. Odometry

Due to extensive use of odometry-based navigation in our algorithms, the following set of experiments were performed to test its reliability and robustness. Precision was evaluated by commanding the robot to turn a specified yaw, driving straight to a defined point or their combination, i.e. rotation towards point and subsequent driving straight. The measured data can be found in Appendix I with the respective plots 17 for the distance measurement and 18 for yaw. It should be noted that the tested algorithm utilized PID control in both parts of the movement thus improving its performance with respect to its rudimentary implementation. Robustness was tested by driving the robot in a specified figure many times and recording its trajectory.

Experimental apparatus consists of: Tape measure Festa $3\,\mathrm{m}$ with the smallest unit being $1\,\mathrm{mm}$ and iPhone 11 with Compass app with negligible error compared to the robot movement.

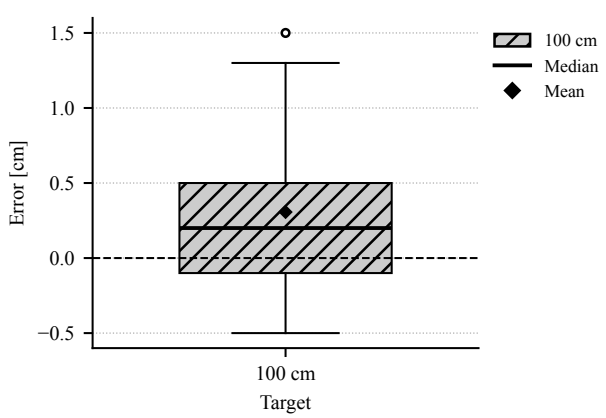

Fig. 17. Odometry distance error. Robot was commanded to travel distance of $1\,\mathrm{m}$ straight. Box plot shows medium, mean values together with outliers marked by circle.

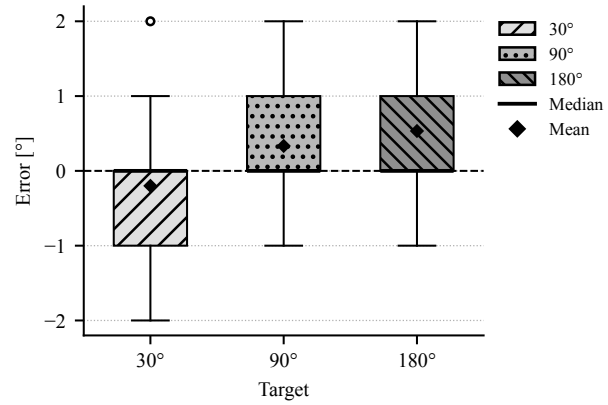

Fig. 18. Odometry yaw error. Robot was commanded to travel turn a specified yaw. Box plot shows medium, mean values together with outliers marked by circle.

In the next part the specified trajectory was a square with $1\,\mathrm{m}$ sides. After circumnavigating the square, robot's offset from starting point was recorded. For each measurement the robot was placed exactly at the same starting point to initialize new recording. Recorded data can be found plotted in Fig. 19 with respective values in Appendix A.

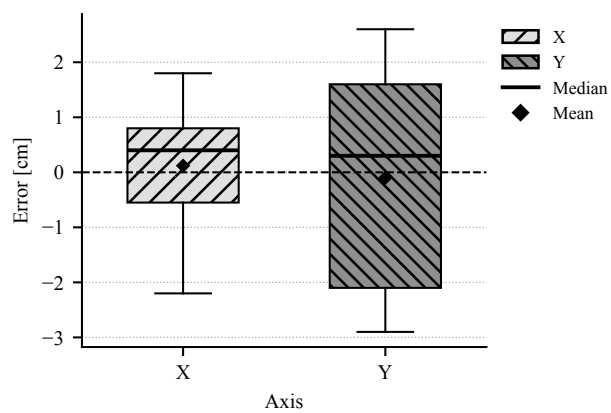

Fig. 19. Robot position offset after encircling square with side lenght of $1\,\mathrm{m}$.

To test odometry robustness the robot circumscribed the same square with $1\,\mathrm{m}$ edge 15 times without mid-movement realignment from the operator. It was observed that after 10 runs the robot started to drift away from the original points and essentially rotated the prescribed square by approximately $10°$ by the end of the test. It should be noted that the robot was commanded with a relatively high forward velocity of $0.7\,\mathrm{m}\cdot\mathrm{s}^{-1}$ during this test, which increased odometry drift due to wheel slippage and other effects. To mitigate this, the proposed algorithm employs a lower velocity of $0.2\,\mathrm{m}\cdot\mathrm{s}^{-1}$. Due to time constraints, the experiment was not repeated at different speeds.

The conducted odometry experiments show low systematic bias and good repeatability for short motions in a structured indoor environment. The mean yaw errors remained below $1°$ in all tested headings, while the $1\,\mathrm{m}$ translational test produced a mean error of only $0.31\,\mathrm{cm}$. Small interquartile ranges show good repeatability. These results indicate that the controller and odometric estimate are sufficiently accurate for

local navigation tasks. However, the square-path experiment revealed cumulative drift over longer trajectories, which is consistent with the known accumulation of odometry errors [9]. Therefore, odometry is suitable for short-range motion execution in the presented task, but its reliability decreases as path length and the number of motion segments increase.

X. Conclusion

This paper demonstrated light-weight, finite-state machine control solution to autonomous robot navigation in a semi-known environment. It uses HSV-based thresholding with further heuristic logic for both green pylon recognition and purple garage pillars recognition. Experiments show that odometry-based movement control is highly accurate for short distances and the robot can reliably return to its home point, where precise parking manoeuvre is performed using purple pillars as reference points.

This algorithm was created and evaluated using TurtleBot number 13. Multiple constants and coefficients were explored and carefully adjusted specifically for this robot.

Html documentation for public methods of the `Algorithm` class and helper functions was generated using pdoc3 [11].

A. Limitations

This solution relies on two key assumptions: there is no obstacle (including the pylon) within 0.8 m of any garage wall, and no obstacle within 0.5 m of the pylon. If these conditions are not met, the robot is likely to collide with such an object, causing the algorithm to fail during execution.

Pylon and garage segmentation is reliable under typical lighting conditions. However, the robot's RGB camera has limitations; for instance, if a pylon is illuminated by direct midday sunlight, the robot may fail to detect it from some angles because it appears white in the camera image. However, even if the robot cannot detect the pylon from one position, it may still be able to identify it from another.

B. Future work

The current set of waypoints used for locating the pylon is adequate for navigating the rectangular test area bordered by a wooden guardrail in laboratory KN:E-130. To operate in a larger environment, additional conditional waypoints would be required. To navigate an environment containing obstacles, further control logic would need to be developed and implemented.

Appendix A

TABLE I
Odometry test results. α denotes target yaw, Δ target distance, and (x, y) denote measured position offsets during $1 \times 1\,\mathrm{m}$ square trajectory. Reported values include sample means μ.

Trial	**Yaw** α [°]			Δ [cm]	x [cm]	y [cm]
	30	90	180	100		
1	32	92	181	101.5	-0.3	-0.3
2	30	89	181	100.1	0.6	1.0
3	30	90	182	100.2	-0.3	1.6
4	28	90	180	99.8	0.5	2.0
5	29	90	180	100.4	0.6	-2.3
6	31	91	180	100.0	1.1	-2.1
7	31	91	180	99.5	-1.1	-2.1
8	29	90	182	100.5	-2.2	1.1
9	30	90	180	101.2	1.0	-2.7
10	30	91	180	100.2	-0.8	2.6
11	30	90	179	100.5	1.8	1.6
12	30	92	181	99.5	0.3	1.0
13	29	90	182	101.3	-1.1	-2.9
14	28	90	180	99.6	1.3	1.2
15	30	89	180	100.3	0.4	2.6
μ	29.8	90.3	180.5	100.31	0.12	0.15

Acknowledgment

This work is based on the *Robolab LAR support package* developed by Libor Wagner, available online at: https://gitlab.fel.cvut.cz/robolab/lar. Various generative AI models were used throughout the development of this work.

Data Availability

The source code and datasets supporting this work are available from the authors upon request via email.

References

[1] eForce Prague Formula, "Autonomous Cars," Czech Technical University in Prague. [Online]. Available: https://eforce.cvut.cz/garage/

[2] T. Baca, M. Petrlik, M. Vrba, V. Spurny, R. Penicka, D. Hert, and M. Saska, "The MRS UAV System: Pushing the Frontiers of Reproducible Research, Real-world Deployment, and Education with Autonomous Unmanned Aerial Vehicles," *Journal of Intelligent & Robotic Systems*, vol. 102, no. 26, 2021.

[3] D. V. Lu, D. Hershberger, and W. D. Smart, "Layered costmaps for context-sensitive navigation," in *Proc. IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS)*, pp. 709–715, Sep. 2014, doi: 10.1109/IROS.2014.6942636.

[4] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.

[5] Python Software Foundation, *math — Mathematical functions*. Python 3 Standard Library. Available: https://docs.python.org/3/library/math.html

[6] Hagberg, A. A., Schult, D. A., Swart, P. J., *Exploring network structure, dynamics, and function using NetworkX*. In Proceedings of the 7th Python in Science Conference, Pasadena, CA USA, 2008, pp. 11–15.

[7] ROS Enhancement Proposal (REP) 103, "Standard Units of Measure and Coordinate Conventions," Available: https://www.ros.org/reps/rep-0103.html

[8] T. Pajdla, *Elements of Geometry for Computer Vision and Computer Graphics*. Prague, Czech Republic: Czech Technical University in Prague, 2025. [Online]. Available: https://cw.fel.cvut.cz/b252/_media/courses/gvg/pajdla-gvg-lecture-2025.pdf

[9] J. Borenstein and L. Feng, "Measurement and Correction of Systematic Odometry Errors in Mobile Robots," *IEEE Transactions on Robotics and Automation*, vol. 12, no. 5, pp. 869–880, Oct. 1996.

[10] Mathcha Team, "Mathcha - Online Mathematics Editor," 2026. [Online]. Available: https://www.mathcha.io

[11] Andrew Gallant, Maximilian Hils, and pdoc3 contributors, *pdoc — Auto-generate API documentation for Python projects*. Version 0.10.0. Available: https://pdoc3.github.io/pdoc/